You said:

Design a Neo4j graph schema for a wine recommendation app using a wine reviews dataset.

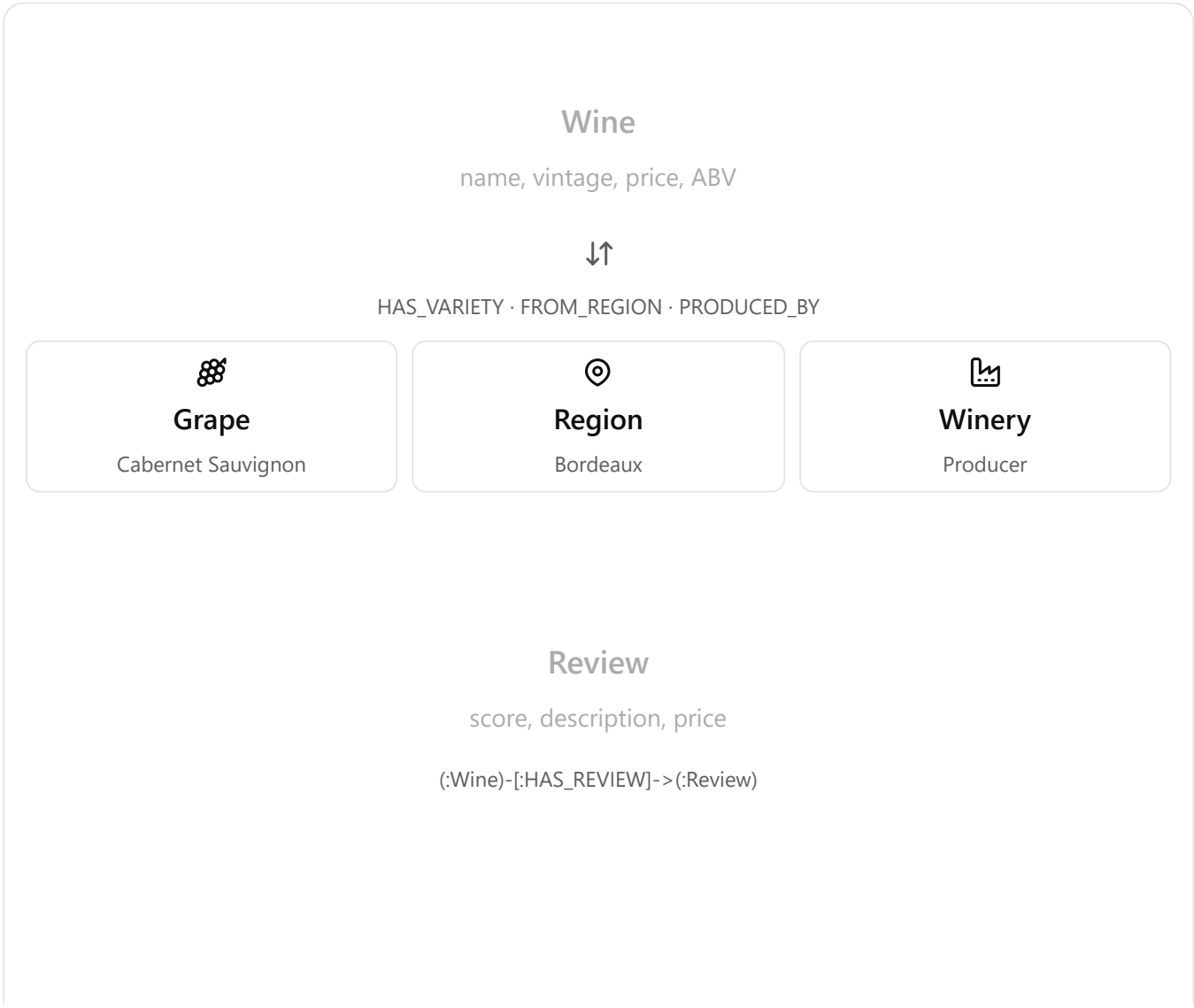
ChatGPT said:

Neo4j graph schema for a wine recommendation app

I'd design the schema around one core idea: **recommend wines by connecting their characteristics, review text, expert ratings, and user preferences.**

A wine reviews dataset—such as one containing wine names, wineries, countries, regions, grape varieties, prices, scores, and written reviews—can populate the wine catalog and its relationships. You can then extend the graph with user interactions to deliver personalized recommendations.

1. Graph data model



User

budget, taste profile

[:RATED] · [:LIKED] · [:WANTS_TO_TRY]

The model has two layers:

- **Catalog graph:** wines, grapes, regions, countries, wineries, reviews, and flavor characteristics.
- **Personalization graph:** users, ratings, likes, purchases, and preferences.

The catalog can be built from the dataset. The personalization layer requires user activity or explicitly collected preferences; it should not be fabricated from critic reviews.

2. Recommended node labels and properties

Node label	Key properties	Purpose
`Wine`	`wineId`, `name`, `vintage`, `price`, `points`, `varietyName`	Main recommendation entity
`Winery`	`wineryId`, `name`	Producer identity
`Region`	`regionId`, `name`	Regional similarity
`Country`	`countryId`, `name`	Geographic filtering
`Grape`	`name`	Grape-based recommendations
`Review`	`reviewId`, `points`, `description`, `price`	Evidence for quality and taste
`TastingNote`	`name`, `category`	Flavor and aroma matching
`User`	`userId`, `budget`, `preferredStyle`	Personalization
`Style`	`name`	Red, white, rosé, sparkling, and other styles

Important modeling decision: keep `Wine` separate from `Review`. One wine may have many reviews, and each review may have a different score, price observation, or description. Do not treat every row in the source dataset as a distinct wine.

If the dataset doesn't include vintage or a reliable wine identifier, use a normalized wine identity based on the available fields, and revisit that key if better catalog data becomes available.

3. Define the relationships

Use relationships to represent meaningful connections rather than storing everything as strings on the wine node.

Relationship	Direction	Meaning
`PRODUCED_BY`	Wine → Winery	Wine's producer
`FROM_REGION`	Wine → Region	Geographic origin
`IN_COUNTRY`	Region → Country	Regional hierarchy
`HAS_VARIETY`	Wine → Grape	Grape composition
`HAS_REVIEW`	Wine → Review	Review evidence
`HAS_NOTE`	Wine → TastingNote	Flavor and aroma
`SIMILAR_TO`	Wine → Wine	Computed similarity
`RATED`	User → Wine	User's rating
`LIKED`	User → Wine	Positive preference
`WANTS_TO_TRY`	User → Wine	Explicit interest
`PREFERS`	User → Style	Preferred wine style

For `SIMILAR_TO`, store properties such as `score` and `method` if you precompute similarities. For user-generated relationships, store a timestamp and any relevant rating or feedback.

Why use separate Review nodes?

Suppose two critics review the same wine, giving it 92 and 96 points. The graph should preserve both reviews rather than overwrite the wine's score.

You can still calculate an aggregate score on the wine node for faster queries:

- `avgPoints`
- `reviewCount`
- `minPrice`
- `maxPrice`

Treat these as derived properties and refresh them when new reviews arrive.

4. Neo4j constraints and indexes

Create uniqueness constraints first. These examples use Neo4j 5+ syntax.

```
CREATE CONSTRAINT wine_id IF NOT EXISTS
FOR (w:Wine)
REQUIRE w.wineId IS UNIQUE;

CREATE CONSTRAINT winery_id IF NOT EXISTS
FOR (w:Winery)
REQUIRE w.wineryId IS UNIQUE;

CREATE CONSTRAINT region_id IF NOT EXISTS
FOR (r:Region)
REQUIRE r.regionId IS UNIQUE;

CREATE CONSTRAINT country_id IF NOT EXISTS
FOR (c:Country)
REQUIRE c.countryId IS UNIQUE;

CREATE CONSTRAINT review_id IF NOT EXISTS
FOR (r:Review)
REQUIRE r.reviewId IS UNIQUE;

CREATE CONSTRAINT user_id IF NOT EXISTS
FOR (u:User)
REQUIRE u.userId IS UNIQUE;

CREATE CONSTRAINT grape_name IF NOT EXISTS
FOR (g:Grape)
REQUIRE g.name IS UNIQUE;
```

Add indexes for common filters:

```
CREATE INDEX wine_name IF NOT EXISTS
FOR (w:Wine) ON (w.name);

CREATE INDEX wine_price IF NOT EXISTS
FOR (w:Wine) ON (w.price);

CREATE INDEX wine_points IF NOT EXISTS
FOR (w:Wine) ON (w.avgPoints);
```

```
CREATE INDEX wine_vintage IF NOT EXISTS
FOR (w:Wine) ON (w.vintage);
```

Use stable IDs rather than names for entities that can share a name. If your source has no IDs, generate deterministic keys during data preparation. Also decide how to handle missing prices, unknown regions, duplicate records, and vintage differences before importing.

5. Load the wine reviews dataset

Assume your CSV has columns such as `title`, `points`, `price`, `country`, `province`, `winery`, `variety`, and `description`. The following is a starting template; adapt the column names to your actual file.

First, normalize the data into a wine catalog and a review file. For example:

- `wines.csv`: one row per distinct wine, with a stable `wineId`.
- `reviews.csv`: one row per review, with `reviewId` and `wineId`.

Then import the catalog:

```
LOAD CSV WITH HEADERS
FROM 'file:///wines.csv' AS row

MERGE (w:Wine {wineId: row.wineId})
SET w.name = row.title,
    w.vintage = CASE
      WHEN row.vintage IS NULL OR row.vintage = ''
      THEN null ELSE toInteger(row.vintage)
    END,
    w.price = CASE
      WHEN row.price IS NULL OR row.price = ''
      THEN null ELSE toFloat(row.price)
    END

WITH w, row
WHERE row.winery IS NOT NULL
  AND row.winery <> ''

MERGE (p:Winery {name: row.winery})
MERGE (w)-[:PRODUCED_BY]->(p);
```

Connect regions, countries, and grapes in separate passes:

```
LOAD CSV WITH HEADERS
FROM 'file:///wines.csv' AS row

MATCH (w:Wine {wineId: row.wineId})

FOREACH (_ IN CASE
  WHEN row.variety IS NOT NULL AND row.variety <> ''
  THEN [1] ELSE [] END |
  MERGE (g:Grape {name: row.variety})
  MERGE (w)-[:HAS_VARIETY]->(g)
)

FOREACH (_ IN CASE
  WHEN row.province IS NOT NULL AND row.province <> ''
  THEN [1] ELSE [] END |
  MERGE (r:Region {name: row.province})
  MERGE (w)-[:FROM_REGION]->(r)
);
```

Import reviews separately:

```
LOAD CSV WITH HEADERS
FROM 'file:///reviews.csv' AS row

MATCH (w:Wine {wineId: row.wineId})

MERGE (r:Review {reviewId: row.reviewId})
SET r.points = CASE
      WHEN row.points IS NULL OR row.points = ''
      THEN null ELSE toInteger(row.points)
    END,
    r.description = row.description,
    r.price = CASE
      WHEN row.price IS NULL OR row.price = ''
      THEN null ELSE toFloat(row.price)
    END

MERGE (w)-[:HAS_REVIEW]->(r);
```

These examples assume one normalized row per wine and a populated wine ID on each review row. In production, use `CALL { ... } IN TRANSACTIONS` for large imports, ensure all uniqueness constraints are created before ingestion, and build a proper region-to-country

hierarchy where source data supports it.

6. Build the recommendation engine

I would start with a content-based recommendation query. It is explainable, easy to test, and works even before you have a large user base.

Example A: Recommend wines similar to a wine the user likes

This query finds wines that share grape variety or region with a reference wine, prioritizes higher-rated candidates, and filters by budget.

```
MATCH (liked:Wine {wineId: $wineId})

MATCH (candidate:Wine)
WHERE candidate.wineId <> liked.wineId
  AND candidate.price <= $maxPrice
  AND candidate.avgPoints >= $minPoints

OPTIONAL MATCH
  (liked)-[:HAS_VARIETY]->(g:Grape)
  <-[:HAS_VARIETY]-(candidate)

WITH liked, candidate,
     count(DISTINCT g) AS sharedGrapes

OPTIONAL MATCH
  (liked)-[:FROM_REGION]->(region:Region)
  <-[:FROM_REGION]-(candidate)

WITH candidate, sharedGrapes,
     count(DISTINCT region) AS sharedRegions

WITH candidate, sharedGrapes, sharedRegions,
     3.0 * sharedGrapes +
     2.0 * sharedRegions +
     coalesce(candidate.avgPoints, 0) / 100.0
       AS recommendationScore

RETURN candidate.name AS wine,
       candidate.price AS price,
       candidate.avgPoints AS points,
       recommendationScore
```

```
ORDER BY recommendationScore DESC
LIMIT 10;
```

This is a baseline, not a complete sensory similarity model. It assumes the `avgPoints` property has already been calculated, and the price and rating filters exclude wines with missing values.

Example B: Calculate aggregate wine scores

After loading reviews, calculate the average score and review count for each wine:

```
MATCH (w:Wine)-[:HAS_REVIEW]->(r:Review)
WHERE r.points IS NOT NULL

WITH w,
     avg(r.points) AS avgPoints,
     count(r) AS reviewCount

SET w.avgPoints = avgPoints,
    w.reviewCount = reviewCount;
```

You can later improve ranking with a Bayesian-adjusted score, which reduces the tendency to over-promote wines with very few reviews.

Example C: Recommend wines based on a user's history

Once users have rated or liked wines, the graph can learn their preferences:

```
MATCH (u:User {userId: $userId})-[:LIKED]->(liked:Wine)
MATCH (liked)-[:HAS_VARIETY]->(g:Grape)
      <-[:HAS_VARIETY]-(candidate:Wine)

WHERE NOT EXISTS {
  MATCH (u)-[:LIKED|RATED|WANTS_TO_TRY]->(candidate)
}

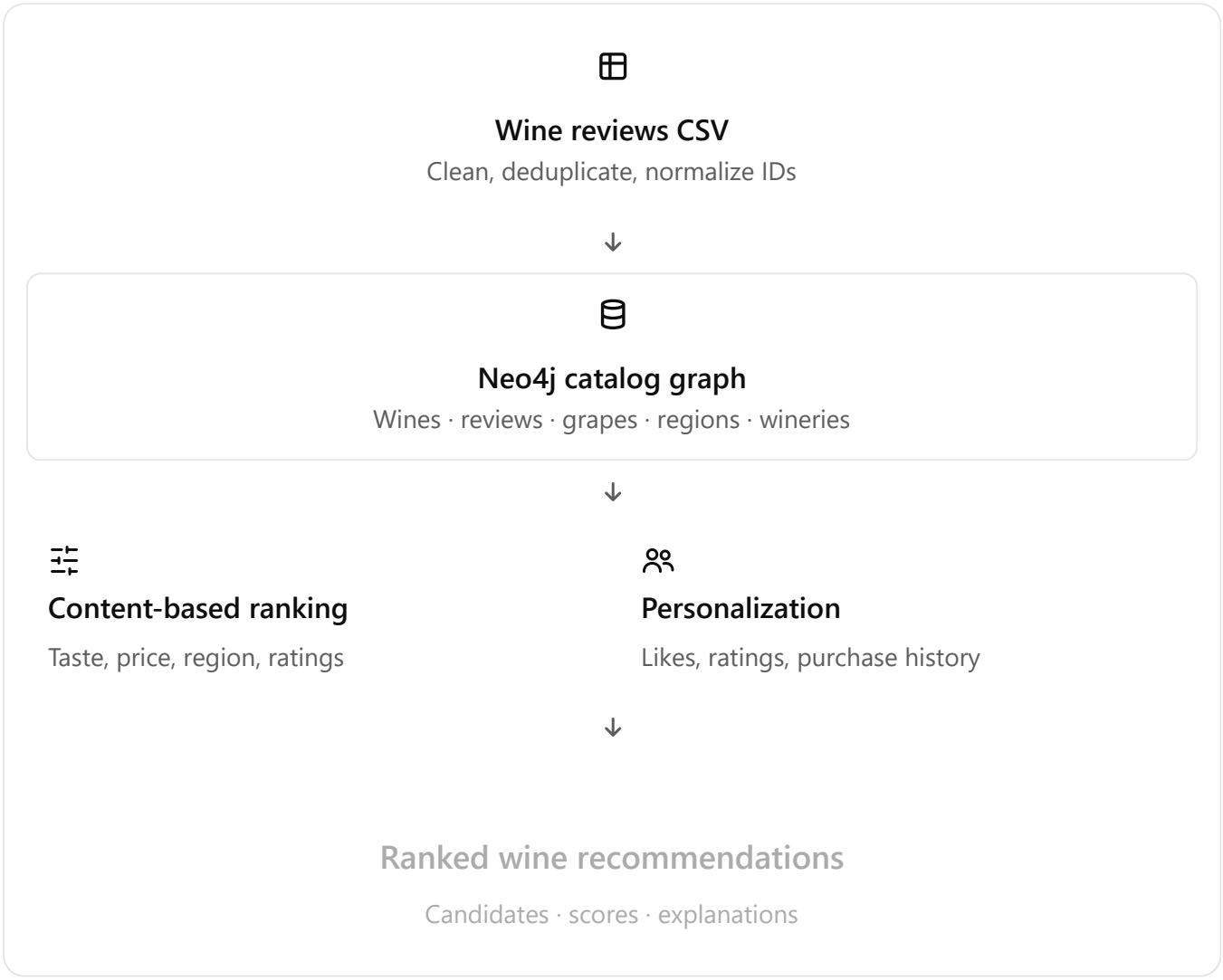
WITH candidate, count(DISTINCT g) AS preferenceMatch
WHERE candidate.price <= $maxPrice

RETURN candidate.name AS wine,
       candidate.avgPoints AS points,
       candidate.price AS price,
       preferenceMatch
ORDER BY preferenceMatch DESC,
```

```
        candidate.avgPoints DESC
LIMIT 10;
```

This is a simple collaborative-personalization starting point. For a more complete implementation, include wines the user has disliked or rated poorly, and weight explicit ratings, purchases, and likes differently.

7. Recommended architecture as the app evolves



For the first release, I recommend:

1. Import wines, reviews, wineries, regions, and grape varieties.
2. Calculate aggregate review statistics.
3. Implement budget-filtered, content-based recommendations.
4. Collect user likes and ratings to personalize results.
5. Add flavor-note extraction from review descriptions.
6. Introduce Neo4j Graph Data Science similarity algorithms or embeddings when you have enough data to justify them.

One final design choice: if your source is the commonly used Wine Enthusiast reviews dataset, it generally contains critic-written reviews rather than consumer preference histories. That makes it excellent for wine discovery and quality-based recommendations, but you will need app-generated user interactions for genuinely personalized recommendations.